1999 such iterations to form parameter distributions. If these distributions are symmetric, we can pretty much just read values straight out of them to form confidence intervals (e.g., the 50th and 1950th values out of 1999 will give us a roughly 95% confidence interval). If they are not, we must do something more complicated, with the best choice being the bias-corrected and accelerated (BCa) approach. Because of the large number of fits that are required, bootstrapping is fairly slow. If the experiment contains many trials, the BCa method makes it even slower (because it incorporates additional "jackknife" resampling, implying one further fitting iteration for almost every trial).[18]

The code accompanying this chapter offers options to generate confidence intervals on fitted parameters. Confidence intervals sometimes imply statistical inference, as for example when they fail to overlap some value and thus imply that our statistic differs significantly from that value. However, in SJ experiments we are more likely to want to ask a question such as whether a particular parameter differs between two conditions for a single observer. To answer this kind of question, you will need to modify or develop the code. If we take the example of whether parameters vary across conditions, my recommendation would be to adopt a permutation test approach.

To do so, take the trials from both conditions and think of each trial as a card in a deck of cards. Making sure you keep each trial intact (i.e., without breaking the link between SOAs and responses) shuffle the trials and then deal them at random into two new piles, each representing a pseudo-condition. If your original conditions contained different numbers of trials, make sure the two pseudo-conditions match the size of the original conditions. For each pseudo-condition, perform a model fit. Now calculate the difference between model parameters in the two pseudo-conditions. This is the value you want to retain. Now repeat this whole process many times. What you are forming is a null distribution of the expected difference between model parameters that would occur just by chance. You can then compare the difference you actually obtained against this null distribution to generate a p value for your difference of interest.

7 Variants of SJ Observer Models

In this chapter, I have presented two variants of a latency-based observer model applied to the SJ task. Both assume that a single SOA will generate an internal response (Δt) that is a Gaussian random variable. Both assume a simple

18 E.g., <SimultaneityNoisyCriteriaMultistart 225–386>. Note that Matlab has inbuilt functions, which could have done most of this *if* you have the statistics toolbox extensions.

1999 such iterations to form parameter distributions. If these distributions are symmetric, we can pretty much just read values straight out of them to form confidence intervals (e.g., the 50th and 1950th values out of 1999 will give us a roughly 95% confidence interval). If they are not, we must do something more complicated, with the best choice being the bias-corrected and accelerated (BCa) approach. Because of the large number of fits that are required, bootstrapping is fairly slow. If the experiment contains many trials, the BCa method makes it even slower (because it incorporates additional "jackknife" resampling, implying one further fitting iteration for almost every trial).[18]

The code accompanying this chapter offers options to generate confidence intervals on fitted parameters. Confidence intervals sometimes imply statistical inference, as for example when they fail to overlap some value and thus imply that our statistic differs significantly from that value. However, in SJ experiments we are more likely to want to ask a question such as whether a particular parameter differs between two conditions for a single observer. To answer this kind of question, you will need to modify or develop the code. If we take the example of whether parameters vary across conditions, my recommendation would be to adopt a permutation test approach.

To do so, take the trials from both conditions and think of each trial as a card in a deck of cards. Making sure you keep each trial intact (i.e., without breaking the link between SOAs and responses) shuffle the trials and then deal them at random into two new piles, each representing a pseudo-condition. If your original conditions contained different numbers of trials, make sure the two pseudo-conditions match the size of the original conditions. For each pseudo-condition, perform a model fit. Now calculate the difference between model parameters in the two pseudo-conditions. This is the value you want to retain. Now repeat this whole process many times. What you are forming is a null distribution of the expected difference between model parameters that would occur just by chance. You can then compare the difference you actually obtained against this null distribution to generate a p value for your difference of interest.

7 Variants of SJ Observer Models

In this chapter, I have presented two variants of a latency-based observer model applied to the SJ task. Both assume that a single SOA will generate an internal response (Δt) that is a Gaussian random variable. Both assume a simple

18 E.g., <SimultaneityNoisyCriteriaMultistart 225–386>. Note that Matlab has inbuilt functions, which could have done most of this *if* you have the statistics toolbox extensions.